DiVA: Dynamic Submap-Level KV Cache Retrieval for Accelerating Visual Autoregressive Models

Jain Koo
Yonsei University
Seoul, Republic of Korea
jain.koo@yonsei.ac.kr

Sungbin Kim
Yonsei University
Seoul, Republic of Korea
sungbin.kim@yonsei.ac.kr

Enhyeok Jang
Yonsei University
Seoul, Republic of Korea
enhyeok.jang@yonsei.ac.kr

Sungwoo Kim
Yonsei University
Seoul, Republic of Korea
sungwoo.kim@yonsei.ac.kr

Seunghyun Lee
Yonsei University
Seoul, Republic of Korea
seunghyun.lee@yonsei.ac.kr

Yunseok Kim
Yonsei University
Seoul, Republic of Korea
yunseok.kim@yonsei.ac.kr

Hyunwuk Lee
Gwangju Institute of Science and Technology
Gwangju, Republic of Korea
hyunwuklee@gist.ac.kr

Yingyan (Celine) Lin
Georgia Institute of Technology
Atlanta, Georgia, USA
celine.lin@gatech.edu

Won Woo Ro
Yonsei University
Seoul, Republic of Korea
wro@yonsei.ac.kr

Abstract

Visual autoregressive (VAR) models utilize coarse-to-fine next-resolution prediction and have emerged as promising alternatives to diffusion models for image generation. Unlike LLMs, VAR generates multiple query tokens at each decoding stage, causing the KV cache to grow rapidly across scales and making inference challenging under limited GPU memory capacity. While prior KV cache retrievals reduce memory burden by offloading the full cache and selectively fetching critical entries, we note that they become suboptimal for VAR in both latency and accuracy, since the criticality estimation process fundamentally changes. To address this, we present DiVA, an algorithm–system co-designed architecture for KV retrieval in VAR models. DiVA shifts the granularity of KV retrieval from tokens to submaps. It uses submap-representative scoring, compressing each key submap into a representative vector to reduce computational overhead while preserving accuracy. To leverage these benefits, DiVA integrates dedicated hardware for submap compression, row-wise scoring, and normalization. DiVA achieves up to 4.23× speedup and 3.02× energy efficiency with only 2.76% area overhead, while preserving generation quality.

1 Introduction

Visual autoregressive (VAR) models [15, 71] have emerged as promising alternatives to diffusion models [35, 42] in image generation, due to their scalability and generality. Unlike conventional autoregressive (AR) models [12, 52], which generate text or images in a next-token prediction manner by sequentially producing a single token at each stage, VAR adopts a coarse-to-fine next-resolution prediction paradigm. In each stage, VAR generates an entire token map at a target resolution, with all tokens within the token map produced in parallel. Thus, the number of decoding steps for high-resolution generation is greatly reduced.

However, this design fundamentally changes the inference characteristics of Transformer-based generation [10, 11, 42] compared with conventional large language models (LLMs) and AR models. As shown in Fig. 1, VAR generates a large number of visual tokens in parallel at each decoding step, and its sequence length grows rapidly as the target resolution increases across scales. Accordingly, unlike next-token AR models [18, 24, 29, 38, 50, 67, 69], where the KV cache size increases linearly [8, 16, 36, 51, 73] as each decoding step appends a single token, VAR appends a large number of KV entries at every step, causing the KV cache size to grow much more rapidly during inference. The resulting KV cache footprint places pressure on GPU memory capacity and makes the practical deployment of large-scale VAR models challenging.

To address the tremendous KV cache size, prior works [22, 30, 37] have explored KV cache retrieval systems that offload the full cache to CPU memory or storage and selectively fetch only critical KV entries [39, 40, 55, 60]. By reducing the size of KV data fetched to the GPU, these systems effectively lower GPU memory usage while preserving generation quality. While these are effective for LLMs, we find that existing KV cache retrieval systems become suboptimal for VAR in terms of both latency and accuracy, as VAR fundamentally changes the criticality estimation process, which evaluates the importance of each KV entry.

During LLM decoding, each step involves a single query vector, and criticality estimation is therefore performed as a GEMV operation. Existing retrieval systems were designed to reduce this GEMV overhead using approximation techniques such as channel reduction [30] and token pooling [20, 54], which reduce the feature dimension or the number of tokens involved in criticality estimation. In contrast, VAR processes a large number of query tokens simultaneously, expanding the same criticality estimation process from a GEMV into a large GEMM operation. Consequently, even after applying these approximations, criticality estimation becomes the dominant performance bottleneck in VAR, accounting for more than 70% of the runtime at later scales, as shown in Fig. 3.

Moreover, these systems also introduce accuracy challenges when applied to VAR, since each query exhibits a distinct importance distribution. Specifically, important channel indices can differ

substantially across rows of the query matrix, while important key tokens can vary across queries of the attention-score matrix. As a result, channel reduction may fail to preserve the critical channels of individual queries, whereas query-key token pooling can blur the distinct key importance of each query, leading to substantial accuracy degradation. These observations reveal that existing KV retrievals are unsuitable for VAR, motivating a new retrieval algorithm tailored to VAR's multi-token generation.

Based on these observations, we propose DiVA, an algorithm-system co-designed architecture for efficient and accurate KV cache retrieval in VAR models. At the algorithm level, DiVA introduces a retrieval method specifically designed for VAR's next-resolution prediction paradigm. Unlike conventional retrieval methods that select individual KV tokens, DiVA elevates the retrieval granularity from tokens to submaps. A submap is a contiguous group of KV tokens generated together at the same resolution scale, which collectively forms a complete spatial token map of the image at that scale. Thus, submap-level retrieval may be better suited to preserving image fidelity, but token-level retrieval may retain only fragmented portions of each spatial token map, disrupting its structural completeness and leading to a drop in accuracy.

To validate this, we analyze the attention patterns of VAR together with the intra- and inter-submap variation in attention scores, as shown in Fig. 6 and 7. Fig. 6 shows a multi-diagonal attention pattern aligned with stage-wise submap boundaries, suggesting that attention is structured around submaps rather than a single token subset. Consistent with this, Fig. 7 shows that tokens within the same submap exhibit similar attention scores, whereas score differences are larger across submaps, indicating the presence of submap coherence. Thus, submap-level retrieval selects important tokens at the granularity of complete, spatially organized token maps, avoiding the fragmentation that may arise from prior finer methods.

Leveraging this submap coherence, DiVA introduces submap-representative scoring, which represents each submap with a single representative vector during criticality estimation. By evaluating importance at the submap level rather than computing scores for all individual KV entries, it substantially reduces the criticality estimation complexity. Moreover, because each submap is stored as a contiguous KV chunk, our algorithm avoids the irregular memory access and sparse gathering overhead that arise in prior finer-granularity retrieval methods. To fully exploit the algorithmic benefit, we design DiVA hardware, a specialized hardware system. DiVA consists of a Submap Compressor, a Row Processing Unit, and a bit-level Normalization Unit, and efficiently executes criticality estimation through a submap-stationary dataflow and channel-wise tiled processing. With this design, the proposed DiVA system effectively eliminates the criticality estimation bottleneck, achieving higher performance than the baselines.

In summary, our main contributions are as follows:

- We observe that existing KV retrieval strategies are suboptimal for VAR because of VAR's inherent multi-query token generation and row-wise divergence in channel and token importance.
- We propose DiVA, an algorithm-hardware co-designed system for KV retrieval algorithm for VAR that converts the retrieval granularity from tokens to submaps and greatly reduces the criticality estimation overhead.
- DiVA achieves up to 4.23× speedup, 3.02× improvement in energy efficiency over baselines, and higher accuracy with only 2.7% chip-area overhead.

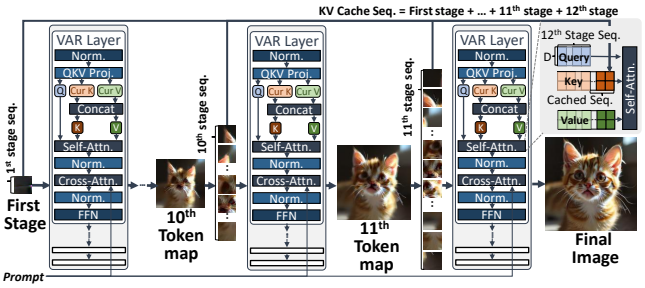


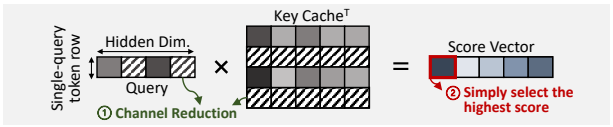
Figure 1: Overall inference process of the VAR model.

2 Background

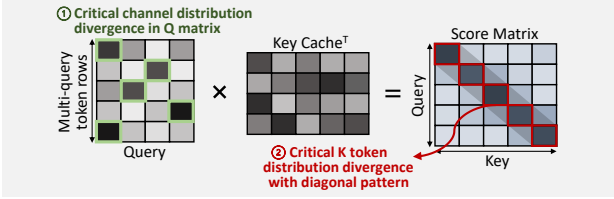
2.1 Visual Autoregressive Modeling

Conventional AR models [4, 7, 26, 27, 45, 61, 65] rely on next-token prediction, which inevitably leads to a large number of decoding steps. This becomes severe for high-resolution image synthesis, where the number of tokens scales with spatial resolution, leading to a large number of autoregressive iterations and high end-to-end latency. In contrast, VAR [15, 53, 56, 59] shifts the generation paradigm from next-token prediction to next-resolution prediction. Motivated by the coarse-to-fine nature of visual content, VAR progressively predicts higher-resolution visual representations (i.e., token maps) based on previously generated lower-resolution ones, reducing the number of autoregressive iterations required for high-resolution synthesis as shown in Fig. 1. VAR decodes an image by a sequence of multi-scale token maps, $\mathbf{R} = (\mathbf{r}_1, \ldots, \mathbf{r}_K)$, where the spatial resolution increases with the scale index k, and $\mathbf{r}_K$ corresponds to the token map of the highest-resolution stage. During inference, when predicting the kth token map $\mathbf{r}_k \in [V]^{H_k \times W_k}$, VAR generates all $H_k \times W_k$ tokens in parallel within the k-th stage, enabling the simultaneous generation of thousands of tokens (e.g., 4096 tokens at the 12th stage in Infinity-8B). This multi-token generation property distinguishes VAR from conventional AR models such as LLMs [4, 14, 26, 27, 45, 46], which typically generate a single token per decoding step.

While this design reduces the number of autoregressive iterations, it introduces distinct inference challenges. VAR still relies on attention over long sequences for high-resolution generation, and it does not use causal masking within each token map, increasing the attention computation for a given sequence length. Moreover, VAR's KV cache grows much faster than in next-token prediction models, as each stage appends all tokens within an entire token map rather than a single token. Thus, VAR faces substantial attention latency and KV cache memory pressure during high-resolution generation.



(a) Criticality estimation in LLM decoding stages.



(b) Criticality estimation in VAR inference stages.

Figure 2: Comparison of the criticality estimation process in (a) LLM and (b) VAR. Darker colors indicate more critical channels/score elements. The diagonal pattern shown in this example reflects a representative attention head pattern observed in VAR, as illustrated in Fig. 6.

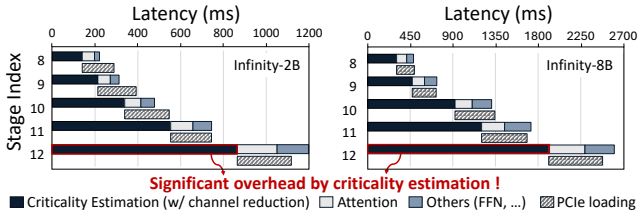


Figure 3: Latency breakdown of prior retrieval systems in Infinity-2B and Infinity-8B with batch size 1. We use the NVIDIA A6000 GPU for evaluation. Three optimization strategies are applied: (1) query channel reduction (30% of the channels are retained), (2) fetching only 30% of the critical KV cache, and (3) layer-wise KV cache prefetching.

2.2 KV Cache Retrieval Systems

To reduce high attention computation cost and large KV cache footprints, recent works [5, 32, 43, 63, 72] have primarily focused on reducing the KV cache size [1, 2, 21, 25, 33, 47, 62, 66, 68]. A straightforward approach is KV cache compression, such as eviction or quantization, which reduces memory usage and attention computation but can introduce irreversible information loss and degrade accuracy.

Another approach is *KV cache retrieval* [22, 30], which preserves the full KV cache in larger memory, such as CPU DRAM [40, 55, 60], and selectively transfers only critical KV entries to GPUs, avoiding the permanent information loss of compression. By restricting attention to this subset, they reduce both computation and memory overhead while largely preserving generation quality and enabling scaling beyond GPU memory capacity. But it introduces a new bottleneck due to the limited PCIe bandwidth, as on-demand KV retrieval can expose data movement latency on the critical path and limit inference performance. To mitigate this, prior works incorporate layer-wise KV prefetching to overlap PCIe transfers with GPU computation, hiding latency from critical paths.

To make KV cache retrieval practical under limited PCIe bandwidth, two key design objectives must be satisfied. First, the criticality estimation stage, which determines the importance of KV entries, must be efficient. As this stage is introduced in addition to the original inference pipeline, it should incur minimal overhead so that selected KV entries can be prefetched to the GPU in time. Second, the criticality estimation must be accurate, since selecting unimportant KV entries can lead to generation quality degradation. Inaccurate estimation often necessitates fetching a larger number of KV entries to maintain quality, which in turn increases PCIe traffic and diminishes the effectiveness of latency hiding through prefetching. Prior works have successfully satisfied these requirements in conventional AR models. However, their effectiveness may not directly carry over to VAR due to its multi-token decoding behavior. This difference may introduce additional challenges for the efficiency and accuracy of criticality estimation in KV retrieval systems.

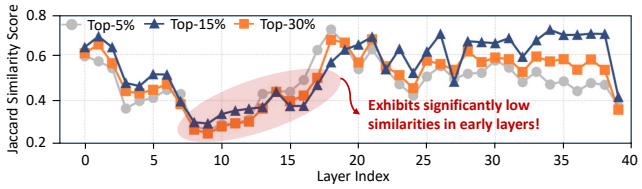
3 Motivation

3.1 Latency Challenges of KV Retrieval in VAR

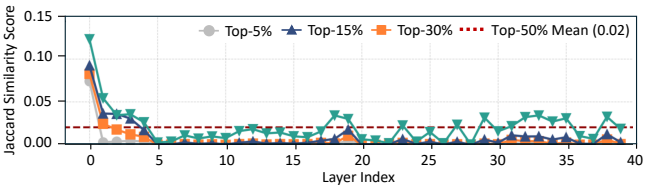
In conventional LLMs [19, 23, 24, 29, 31, 44, 58], each decoding step generates only a single query token, allowing critical KV entries to be identified through a GEMV operation between the query vector and the cached key matrix, as shown in Fig. 2a. Although this GEMV operation introduces non-negligible overhead, prior KV cache retrieval methods effectively reduce its cost through optimization techniques such as channel reduction [30] and token pooling [20]. Channel reduction retains only a subset of high-magnitude feature dimensions, whereas token pooling aggregates multiple query and key tokens into coarse representatives to reduce the effective sequence dimensions. These techniques decrease the feature and sequence dimensions of criticality estimation, respectively, making the overhead manageable in single-query LLM decoding. However, this overhead is significantly amplified in VAR due to its multi-query generation pattern.

Each VAR decoding step generates $H_k \times W_k$ query tokens in parallel, fundamentally changing the criticality estimation. Instead of executing a single query through GEMV, VAR multiplies an entire query matrix against the cached key matrix, transforming the operation into a large GEMM, as shown in Fig. 2b. Even after applying optimizations developed for prior KV cache retrievals, criticality estimation latency increases substantially as its cost scales with the number of query tokens generated at each scale, reaching 4096 simultaneous queries in Infinity-8B.

To quantify this effect, we analyze the per-stage latency breakdowns of Infinity-2B and 8B with InfiniGen, which uses channel reduction to reduce estimation latency, as shown in Fig. 3. Despite applying several methods to mitigate the latency of the criticality estimation stage, this stage still accounts for more than 71% of the end-to-end execution time at the final stage in both models. This large estimation latency lies directly on the critical path, substantially increasing the total execution time. As shown later, other approaches such as token pooling also fail to fully eliminate the criticality estimation overhead (see Fig. 10).



(a) Query-wise critical channel index similarity in the query matrix.



(b) Query-wise critical token index similarity in the score matrix.

Figure 4: Evaluation of the Jaccard similarity between (a) critical channel indices identified from multiple query rows in the query matrix and (b) critical token indices identified from multiple query rows in the score matrix, across layers under various top-k settings.

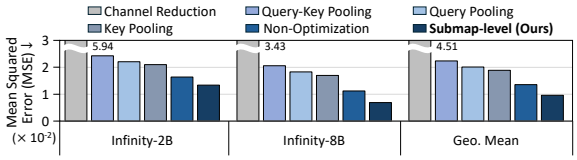


Figure 5: MSE of the attention output matrix under different criticality estimation mechanisms, at the same 35.75% retrieval budget, measured against full-KV attention. Non-optimization computes criticality estimation with exact $QK^\top$ and retrieves the top-35.75% tokens, whereas the reference preserves all KV tokens. Channel reduction retains 30% of the channels, and the pooling size is 32. Lower values indicate higher similarity.

3.2 Accuracy Challenges of KV Retrieval in VAR

3.2.1 Row-wise channel distribution divergence. Beyond the latency overhead, VAR's multi-query generation also introduces a fundamental accuracy challenge for channel reduction. In LLM decoding, a reduced channel subset can be easily selected for a single query vector. In VAR, however, the important high-magnitude channels can vary across rows of the query matrix, as illustrated in Fig. 2b ①. This row-wise channel-importance divergence makes it difficult to identify a single reduced channel subset that accurately serves all queries. Thus, channels that are critical to certain query rows may be omitted from criticality estimation. As these omitted channels with large magnitude substantially contribute to the corresponding criticality scores, their exclusion may distort the relative ranking of KV entries and degrade retrieval accuracy.

To quantify this channel divergence, we measure the similarity between the important-channel sets selected by different query rows using the Jaccard score [6], $\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$, which is widely used to measure the index similarity between two sets. The Jaccard score can evaluate whether different query rows select similar channel subsets or not. For all layers in the later stages (i.e., 8th to 12th stages), we vary the degree of channel reduction by selecting the top-30%, 15%, and 5% channels for each query row, and measure the similarity of the resulting channel index sets as shown in Fig. 4a. In Fig. 4a, the Jaccard score remains substantially low, with an average of 0.49 across layers and staying below 0.26 in early layers. This limited overlap confirms that important-channel sets differ substantially across query rows. Thus, selecting a single channel subset shared across all queries can omit channels that are critical to individual rows, leading to accuracy degradation.

3.2.2 Row-wise token distribution divergence. The divergence observed in the channel dimension also appears along the token dimension. In LLM decoding, token selection operates on a single score vector, as depicted in Fig. 2a ②. In contrast, VAR produces a score matrix with multiple query rows, where each row can exhibit a markedly different score distribution, as shown in Fig. 2b ②. As KV cache retrieval requires materializing a single subset of key tokens shared across all queries, and fetching it to the GPU before attention execution, this row-wise token divergence makes retrieval challenging. Different query rows can exhibit different key preferences, making it difficult to determine a single subset.

To quantify this behavior, we measure the Jaccard similarity of the key index sets selected from different query rows in the score matrix under various top-k settings, similar to Fig. 4a. By varying the retrieval budget (i.e., the number of KV entries selected for retrieval), we evaluate whether different query rows consistently prefer similar key subsets. As shown in Fig. 4b, the key index sets selected by different query rows exhibit substantially low Jaccard similarity, with an average of 0.02 across layers. This low similarity indicates that selecting a single key subset across all query rows, which is required in KV retrieval, is challenging and can degrade the accuracy.

To further evaluate how the low similarity caused by diversity affects attention accuracy, we apply each method to reduce criticality estimation overhead and measure the mean squared error (MSE) of the resulting attention outputs, as shown in Fig. 5. Evaluated schemes include channel reduction to assess the impact of channel-importance diversity, and key pooling to assess key-token diversity. We also add query and query-key pooling to examine the effect of aggregating queries.

These schemes exhibit substantially higher MSE than Non-optimization (the design without optimization for reducing criticality estimation overhead), confirming that the row-wise diversity of channel and token importance limits the ability of prior optimization methods to preserve accurate attention outputs in VAR. In particular, query-side pooling is unsuitable for VAR because it merges query rows without considering their distinct key preferences. The pooled representation therefore fails to preserve the preference of each individual query, causing important keys for some queries to be omitted.

4 Submap-Level Retrieval

We turn our attention to a granularity tailored to VAR, which we refer to as a *scale-submap (i.e., submap)*, and analyze the design space of submap-level KV retrieval. A *submap* is the contiguous chunk of

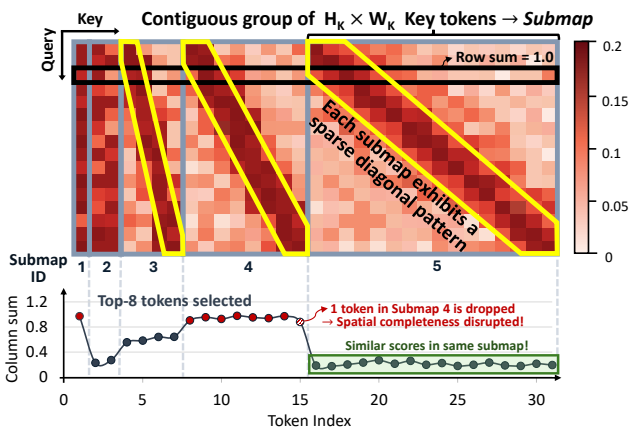


Figure 6: Heat-map visualization of the probability matrix (P) of the 23rd head in the first layer of Infinity-8B. Submap refers to the contiguous group of tokens within the grey boundary.

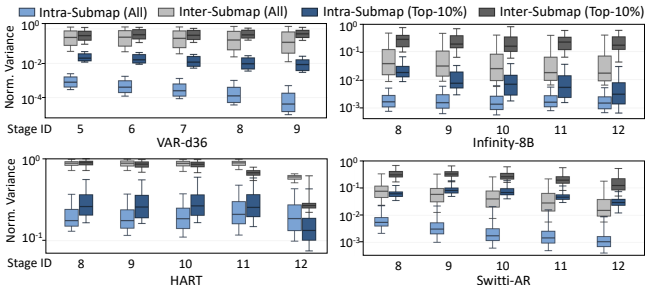


Figure 7: Token-wise score variance across stages (y-axis in log scale) for intra-/inter-submap in VAR-d36, Infinity-8B, HART, and Switti-AR, in all heads and top-10% heads with the weakest intra-submap coherence in their aggregated key scores. The center line in each box denotes the mean, the box spans the 30%–70% percentiles, and the whiskers span the 20%–80% percentiles.

$H_k \times W_k$ key and value tokens produced together at each resolution scale $\mathbf{r}_k \in [V]^{H_k \times W_k}$ of VAR. As all tokens at each resolution jointly form a single complete token map, this unit represents a full image at that specific resolution. Visually, each submap corresponds to the entire KV tokens separated by the grey boundary lines in Fig. 6.

4.1 Submap Coherence Property

As each submap corresponds to a group of KV tokens generated together in VAR's next-resolution decoding process, we hypothesize that KV retrieval at the submap granularity may better align with the underlying attention structure than finer-grained retrieval methods. To examine this, we first analyze the attention patterns. In Fig. 6, the attention probability matrix exhibits a sparse multi-diagonal pattern consistent with the observations in prior work [32, 43], in which attention mass is concentrated along multiple narrow diagonal bands. Our analysis in Section 3.2.2 reveals that row-wise token divergence (Fig. 2b ②) makes this pattern challenging for prior token-level methods [32, 43], since a single shared subset cannot reliably preserve the distinct key preferences of all queries. In contrast, we turn our attention to the alignment between the multi-diagonal pattern and the scale-wise submap structure. This alignment indicates that VAR's attention structure and its token-wise divergence are organized at the submap level. We therefore consider this same pattern not as a challenge but as an opportunity to exploit VAR's submap-aligned structure.

To assess this opportunity, we analyze the column-wise sum, which serves as an aggregated measure of key importance, as shown at the bottom of Fig. 6. As high-score regions shift diagonally, aggregating the scores across queries distributes their high-score contributions evenly across the key tokens within each submap. Thus, key tokens within the same submap exhibit similar aggregated importance, whereas importance differs relatively more across submaps. This intra-submap coherence implies that tokens within the same submap may be considered collectively, providing a basis for criticality estimation and retrieval at the submap granularity.

To quantify this submap coherence, we analyze the intra- and inter-submap variance of key token importance across decoding stages, as shown in Fig. 7. Across all evaluated models and stages, normalized intra-submap variance remains consistently lower than inter-submap variance, with an average difference of 80.57%. This trend persists even among the 10% of attention heads exhibiting the weakest intra-submap coherence, identified by the largest variance ratio, $R_h = \frac{\text{var.}^{\text{intra}}(h)}{\text{var.}^{\text{inter}}(h)}$. Even for these heads, intra-submap variance remains 63.96% lower than inter-submap variance on average across models. These results indicate that submap coherence is a structural characteristic of VAR's next-resolution decoding rather than a pattern confined to a small subset of attention heads.

This coherence property can be understood from VAR's scale-wise decoding process. At each scale, VAR jointly generates $H_k \times W_k$ tokens to form a complete spatial token map. Each token therefore corresponds to a specific location in the underlying 2D grid, and together, these tokens preserve the spatial organization of the image at that scale. A submap is therefore not merely a contiguous group of keys in the KV cache, but a spatially organized structural unit, as illustrated in the left green box of Fig. 8. Submap-level retrieval preserves these spatial relationships, whereas finer-grained retrieval may retain only a fragmented portion of the token map, as illustrated in the left red box of Fig. 8. For example, as illustrated at the bottom of Fig. 6, top-k token-level retrieval with k=8 selects seven tokens from the 4th submap and one from the 1st submap. Thus, one token from the 4th submap is omitted, preventing the corresponding submap from being preserved as a complete spatial token map.

Fig. 5 provides further evidence for the importance of preserving the spatial completeness of each submap. Submap-level selection (Ours) achieves lower attention-output error than exact token-level selection (Non-Optimization), which directly retrieves the tokens with the highest aggregated attention scores. This result indicates that preserving the complete spatial structure of a submap by submap-level selection can be more important than selecting individual high-importance tokens, motivating a retrieval design that treats each submap as a single selection unit. Consistently,

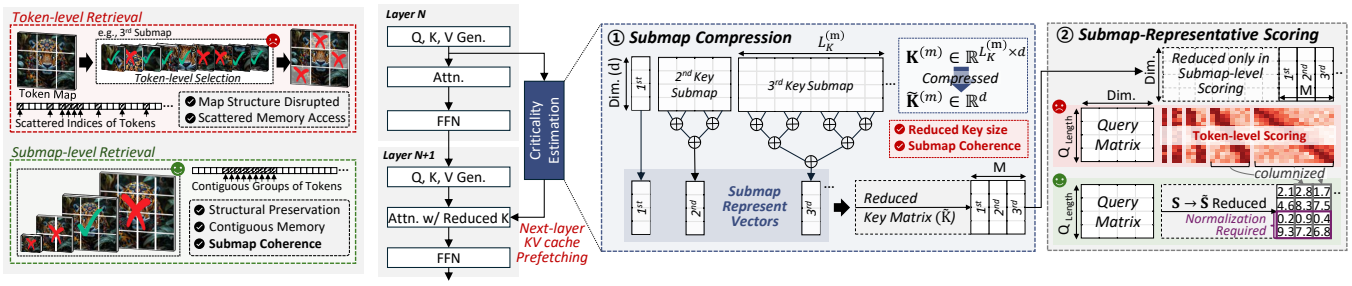


Figure 8: Comparison of token-level and submap-level retrieval (left) and overview of the submap scoring pipeline (right).

complete-submap retrieval achieves higher GenEval scores than finer spatial units under identical full-attention-based scoring and matched KV budgets (Fig. 15b).

4.2 Benefits of Submap-Level Retrieval

Computational efficiency. Although finer-grained methods reduce the effective key-token length through pooling or clustering, their groups are typically much smaller (e.g., 16/32/64 tokens) than a complete submap. Thus, they still require a large number of finer-grained scores, retaining substantial computational complexity and failing to effectively eliminate criticality estimation overhead. In contrast, since all tokens in a submap are selected together, criticality estimation only needs a single score for each submap rather than individual token-level scores. This reduces the number of scores to determine from L_K tokens to only M submaps (e.g., from 10,521 to 13 at the final stage of Infinity-8B), greatly reducing the scoring complexity and criticality estimation overhead.

Regular memory access pattern. Finer-grained methods often incur irregular memory access patterns and require sparse gathering from non-contiguous memory locations, introducing extra overhead during KV cache access. In contrast, since each submap is stored as a contiguous KV cache chunk, retrieval can be performed over contiguous memory regions, avoiding irregular access and sparse gathering overhead. Although whole-submap retrieval may fetch unnecessary tokens within each selected submap in some cases, our analysis reveals that its performance impact remains minimal because of DiVA's contiguous access.

Retrieval accuracy. Simply increasing the group size in prior methods can reduce computation and irregular memory access, but introduces a fundamental limitation. Because groups are formed without leveraging submap boundaries, a single group may contain tokens from multiple submaps, though token importance can substantially vary across submaps. Therefore, these group-wise selection methods may either fetch low-priority tokens, reducing retrieval efficiency, or discard important tokens, degrading generation quality. Fig. 15b further shows that submap-level selection better preserves generation quality than other group-wise selections, highlighting the benefit of leveraging submap structure.

Overall, by aligning retrieval with VAR's inherent submap structure, submap-level retrieval avoids the conventional granularity trade-off, where finer units improve selection precision but retain high scoring complexity, whereas coarser units reduce complexity at the expense of accuracy. It greatly reduces scoring complexity without sacrificing accuracy and also enables regular memory access.

5 DiVA

5.1 Submap Scoring Pipeline

Building on the demonstrated benefits of submap-level selection, we propose *S2MR* (Scale-Submap Granularity Retrieval), an efficient KV retrieval algorithm. As all tokens in a submap are selected together to preserve the spatial structure and completeness of the corresponding token map, criticality estimation only needs to determine which submaps should be fetched. A straightforward approach would be to construct the full score matrix using a large GEMM, aggregate the token-level scores within each submap, and rank the submaps based on their aggregated scores. Although this approach performs retrieval at the desired granularity, it remains inefficient as it explicitly computes scores for every key token, even though only one importance score is ultimately needed for each submap. To address this inefficiency, our scoring reformulates criticality estimation from token-level scoring to direct submap-level scoring by using representative vectors that preserve the aggregated score of each submap.

In detail, representative-based scoring first compresses each key submap before criticality estimation (i.e., submap compression). Let the mth key submap be represented as $\mathbf{K}^{(m)} \in \mathbb{R}^{L_K^{(m)} \times d}$, where $L_K^{(m)}$ is the number of tokens in the mth submap and d is the hidden dimension. It aggregates these tokens into a single representative vector as $\widetilde{\mathbf{K}}^{(m)} = \sum_{\ell=1}^{L_K^{(m)}} \mathbf{K}_\ell^{(m)} \in \mathbb{R}^d$, as Fig. 8①. M representative vectors are then concatenated within each attention head to form $\widetilde{\mathbf{K}} \in \mathbb{R}^{M \times d}$, where M denotes the number of submaps. Then, submap-representative scoring is performed as $\mathbf{Q}\widetilde{\mathbf{K}}^\top$ as Fig. 8②, reducing the scoring GEMM from $(L_Q, d) \times (d, L_K)$ to $(L_Q, d) \times (d, M)$. This reduces the effective key token length from L_K to M, lowering the complexity by a factor of L_K/M (e.g., 809× reduction in the final stage where $L_K = 10521$ and $M = 13$).

Since submaps contain different numbers of tokens, directly comparing their summed scores can bias larger submaps. To address this, we apply length-aware normalization by first dividing the representative score by the submap length and then adding a logarithmic correction based on its length as $\widetilde{S}_{i,m} = \frac{\mathbf{q}_i^\top \widetilde{\mathbf{K}}^{(m)}}{L_K^{(m)}} + \log_2 L_K^{(m)}$. This length-aware normalization accounts for the aggregated attention mass contributed by each submap. Assuming similar token-level

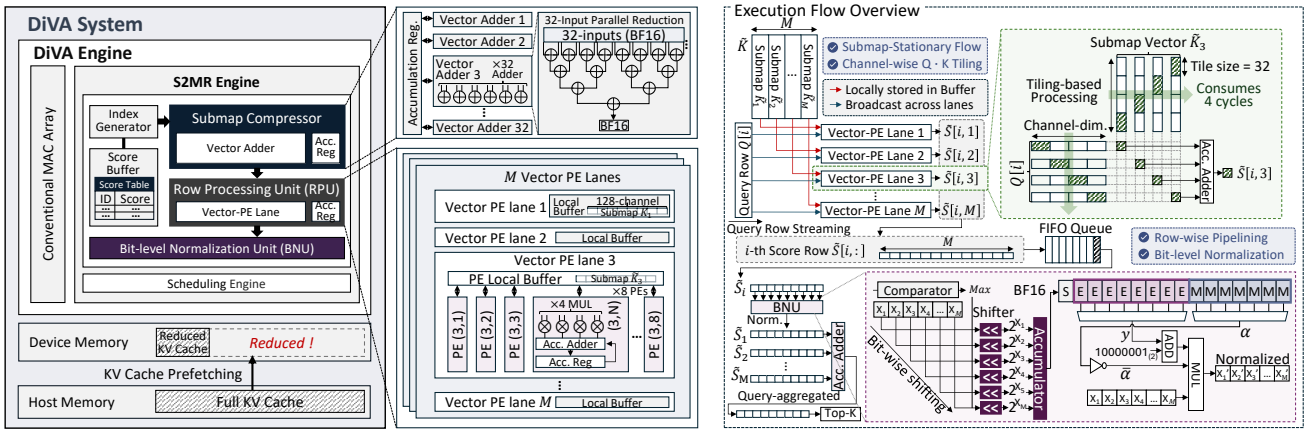


Figure 9: Overview of the DiVA system architecture (left) and its execution flow (right). The S2MR Engine is integrated within DiVA and consists of the Submap Compressor, Row Processing Unit, Bit-level Normalization Unit, and Scheduling Engine.

Table 1: Hardware configuration of DiVA (e.g., $M = 13$)

MAC Array	32-lane BF16 MAC Unit × 1344 per MAC Unit: 32 Mul, 32 Add, 32 Acc. Reg.
Submap Compressor	32 vector adders (32-ch BF16) per Vector Adder: 32-input Adder Tree 32 Acc. Reg. (i.e., Accumulation Register)
RPU	M Vector-PE lanes (32-wide) per lane: 8 PEs (w/ 4 MUL, 4 ADD, 1 Acc. Reg.)
BNU	M Shifter / M Sub / M Mul / $M-1$ Adder $M-1$ Comparator / 1 Inverter

scores, this mass can be approximated as $\log_2 \sum_{\ell=1}^{L} 2^{z_\ell} \approx \bar{z} + \log_2 L$, where $\bar{z}$ is the mean token-level score. This approximation is exact when all scores are identical. The base-2 formulation is further aligned with DiVA's bit-level Normalization Unit (BNU) in Section 5.4, enabling consistent and hardware-efficient normalization.

As a result, representative scoring produces a compact score matrix $\widetilde{\mathrm{S}} \in \mathbb{R}^{L_Q \times M}$, instead of the large token-level score matrix $\mathrm{S} \in \mathbb{R}^{L_Q \times L_K}$. Each of the M columns ($\widetilde{\mathrm{S}}^{(m)}$) of the compact score matrix $\widetilde{\mathrm{S}}$ is the compressed score vector of each submap. It then normalizes each query row to account for variations in score magnitude across query rows, and aggregates the normalized scores across query rows to obtain a single score per submap as $\widetilde{\mathrm{S}}_{\mathrm{agg}} \in \mathbb{R}^{M}$ (the detailed execution flow is introduced in Section 5.4).

The submaps are then ranked according to $\widetilde{\mathrm{S}}_{\mathrm{agg}}$, and the highest-ranked submaps are fetched to the GPU. This reformulation reduces criticality estimation overhead by replacing token-level scoring with compact submap-level scoring, while preserving complete spatial token maps during KV retrieval.

5.2 Opportunities of DiVA Hardware System

Although our scoring pipeline theoretically reduces the cost of submap-level criticality estimation by greatly reducing the scoring dimension, its benefits may not be fully realized in general-purpose accelerators. To reduce the effective key-side token length, it performs submap compression, which aggregates all token vectors within each key submap. This large-scale vector reduction may be less efficiently executed on general-purpose MAC arrays, which are primarily optimized for dense matrix multiplication. Thus, this mismatch reveals an opportunity to consider specialized hardware that executes submap aggregation with higher efficiency.

Beyond submap compression, S2MR transforms scoring into an asymmetric $(L_Q, d) \times (d, M)$ multiplication, where M is far smaller than L_Q (L_Q is 315× larger than M in later scales). When the small output dimension M falls below the tile width of conventional dense-matrix engines, it may lead to partial tile utilization or zero-padding, potentially wasting compute cycles and degrading energy efficiency. The workload also exhibits an imbalance in operand-reuse patterns. The small set of submap vectors is reused across all L_Q query rows, whereas each query row is reused across only M vectors. Although general-purpose dataflows can exploit operand reuse, fully leveraging this imbalanced reuse pattern requires specialized dataflow. These characteristics motivate the design of a specialized scoring engine and dataflow that exploit the small output dimension and asymmetric operand reuse.

Finally, since score magnitudes vary across the rows of the score matrix $\widetilde{\mathrm{S}} \in \mathbb{R}^{L_Q \times M}$ as shown in Fig. 8, row-wise normalization is required. S2MR does not require exact Softmax but only needs to preserve the coarse-grained priority ordering among submaps, allowing normalization to be implemented using substantially cheaper operations without costly exponential units. Moreover, since each score row contains only M elements, all elements can be processed fully in parallel with minimal area overhead. Together, these properties expose an opportunity to design a lightweight normalization unit tailored to S2MR.

5.3 DiVA Hardware Architecture Overview

DiVA consists of a CPU with high-capacity host memory, low-capacity device memory, and the DiVA Engine (Fig. 9). The full KV cache is offloaded to host memory, while device memory temporarily holds the fetched KV submaps selected after criticality estimation. DiVA integrates a conventional MAC array for the primary non-retrieval computations (e.g., self-attention, FFN layers), while offloading KV retrieval computations to the S2MR Engine, which consists of a Submap Compressor, Row Processing Unit, Bit-level Normalization Unit, and Scheduling Engine.

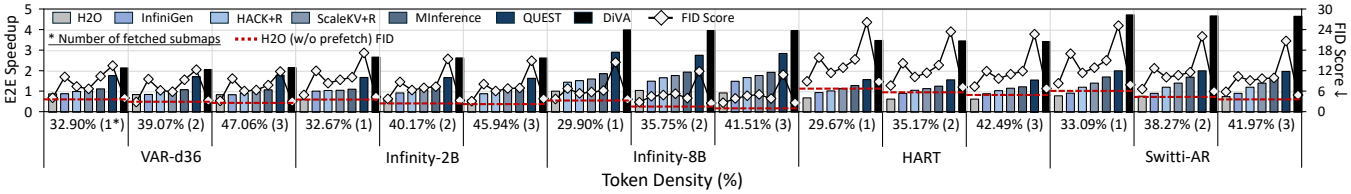


Figure 10: Speedup (normalized to Full KV Fetch) and FID across retrieval densities. FID is computed against real ImageNet images for VAR-d36 and against Full KV Fetch outputs for the other models.

5.4 Hardware Design and Execution Flow

MAC Array. The MAC array performs Transformer computations (linear projection, attention, FFN) outside the retrieval path, which decouples the retrieval process and enables DiVA to specialize the retrieval path for submap prioritization.

Index Generator and Score Table. Index Generator fills the table entry with each submap index as (batch ID, head ID, submap ID) in the Score Table within the Score Buffer. After criticality estimation, the representative score of each submap is written to its corresponding index entry.

Submap Compressor. The Submap Compressor aggregates all tokens within each key submap into a single representative vector $\tilde{K}^{(m)}$. The compressor consists of a BF16 adder array organized as adder-tree structures with accumulation registers. It contains 32 vector adders, allowing it to process 32 key-token vectors across 32 channel elements in parallel during each reduction cycle. Accumulation registers store intermediate partial sums and carry them forward across successive reduction cycles until the final vector is produced. As key tokens of a submap are generated by the MAC array after linear projection and written back to device memory, the compressor concurrently aggregates them into a representative vector, fusing compression into the write-back path and eliminating redundant aggregation and full KV cache access across steps.

Row Processing Unit (RPU). As introduced in Section 5.2, the resulting score computation $(L_Q, d) \times (d, M)$ has a highly asymmetric shape and imbalance in its operand-reuse pattern. To exploit these, RPU adopts a submap-stationary dataflow that stores the highly reused M submap vectors in PE-local buffers while streaming query rows. As each submap vector occupies only 256 B in BF16, all M vectors can be stored in their respective local buffers throughout the processing of the entire query stream within each attention head. This maximizes local data reuse and eliminates repeated memory accesses to load submap vectors. Within this flow, each lane computes the dot product between one streamed query row and one stationary submap vector. In detail, RPU assigns each of the M submap vectors to a dedicated vector processing element (Vector-PE) lane. Each lane contains 32 MAC units and processes the 128-channel dimension over four 32-channel tiles. For each streamed query row, the same query tile is broadcast to all M lanes, while each lane multiplies it by the corresponding tile of its assigned submap vector and accumulates the partial sums locally in the accumulation register. After 4 cycles, the lanes independently produce the M submap scores, which are directly assembled into the complete score row $\tilde{S}[i, 1:M]$, without requiring inter-lane data movement. As query rows are continuously streamed via RPU, the stationary submap vectors remain in the local buffers and are repeatedly reused across all passes. RPU generates a complete score row and immediately forwards it to the subsequent stage without waiting for the remaining rows, forming a tightly coupled row-streaming pipeline.

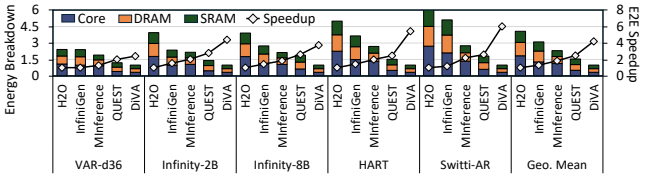


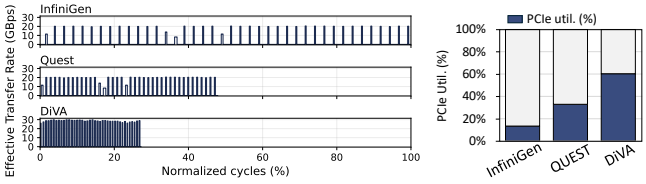
Figure 11: Comparison of energy consumption breakdown and speedup. Energy breakdown is normalized to DiVA, and speedup is normalized to H2O.

Bit-level Normalization Unit (BNU). Each score row produced by RPU must be normalized before query-wise aggregation, as score magnitudes vary across query rows (Fig. 8), which causes high-magnitude rows to dominate the aggregated score, leading to inaccurate submap selection. Building on Softermax [48], BNU replaces base-e exponentiation with base-2 shifting: $p_i = \frac{2^{s_i - \max(s)}}{\sum_{j=1}^{M} 2^{s_j - \max(s)}}$. As the row length is reduced to M, this allows BNU to fully parallelize bit shifting operations across all row elements with minimal area overhead. To further simplify division, BF16 denominator is represented as $(1+\alpha)2^y$, yielding $\frac{2^{s_i - \max(s)}}{(1+\alpha)2^y} \approx (1-\alpha)2^{s_i - \max(s) - y}$. The term $(1-\alpha)$ is obtained with a bit inverter, eliminating a general-purpose divider. Each normalized row is immediately accumulated into a running M-element score vector, avoiding materialization of the full $\widetilde{S} \in \mathbb{R}^{L_Q \times M}$ matrix. After all query rows are processed, the submaps with the highest scores are selected for retrieval.

Scheduling Engine. The Scheduling Engine orchestrates the DiVA system. First, it schedules next-layer KV prefetching to overlap PCIe transfers with GPU computation, enabling intra-stage latency hiding, as in conventional retrieval systems. Second, it organizes the fine-grained execution of a 3-stage row-wise streaming pipeline, which consists of row-wise scoring, normalization, and aggregation, so that the stages are tightly overlapped. This row-streaming organization is critical for latency reduction as (1) it eliminates the data movement of the entire intermediate score matrix and (2) allows scoring and normalization to proceed in a pipelined manner, which avoids the stall inefficiency arising from inter-row processing time imbalance.

Table 2: Quantitative comparisons on the GenEval and ImageReward benchmarks when the fetched submap count is 2. 2Obj: Two Objects, Pos: Position, CAttr: Color Attribution, All: Overall GenEval score. Across 4 seeds, the standard errors are about 0.3 (All), 1.0 (2Obj), 1.1 (Pos), 0.9 (CAttr), and 0.01 (ImageReward).

Model	Infinity-2B					Infinity-8B					HART					Switti-AR				
Metric	GenEval				ImageReward	GenEval				ImageReward	GenEval				ImageReward	GenEval				ImageReward
Method	2Obj	Pos	CAttr	All	Avg. Score	2Obj	Pos	CAttr	All	Avg. Score	2Obj	Pos	CAttr	All	Avg. Score	2Obj	Pos	CAttr	All	Avg. Score
Full KV	81.6	36.0	57.3	68.7	1.111	95.5	56.3	75.0	82.0	1.352	68.3	16.9	41.4	55.0	1.088	74.9	23.2	50.9	63.0	0.991
H2O	80.8	35.8	57.0	68.3	1.103	95.0	55.8	74.0	81.6	1.347	68.1	15.9	41.3	54.2	1.083	74.1	23.7	49.9	62.4	0.986
InfiniGen	77.5	25.3	52.0	65.5	1.025	92.2	45.3	65.8	76.9	1.279	65.9	13.3	38.8	52.5	1.032	73.8	21.7	48.1	60.9	0.938
HACK+R	78.0	29.8	53.0	66.9	1.089	93.2	48.8	68.3	77.8	1.306	66.1	14.0	38.9	52.8	1.075	74.1	22.8	49.1	61.9	0.953
ScaleKV+R	78.0	29.3	52.0	67.0	1.089	92.9	50.0	69.0	78.1	1.290	66.3	13.8	38.8	52.9	1.079	74.4	22.1	48.7	61.6	0.951
MInference	78.0	27.0	51.5	66.3	1.081	92.2	47.8	66.5	77.1	1.294	66.2	13.5	38.6	52.6	1.059	73.9	21.9	48.8	61.1	0.939
Quest	75.3	23.8	50.0	64.2	0.993	88.4	44.8	65.0	76.0	1.260	64.9	13.0	38.4	51.9	1.006	73.3	21.5	47.6	60.6	0.921
DiVA (ours)	80.6	35.8	57.8	68.5	1.111	94.4	55.5	74.3	81.9	1.349	68.5	16.3	40.9	54.8	1.086	74.8	23.5	50.5	62.7	0.987



(a) Effective PCIe data transfer rate. **(b) PCIe utilization.**

Figure 12: Comparison of effective PCIe data transfer rate and PCIe utilization across InfiniGen, QUEST, and DiVA.

Table 3: Area and power breakdown of DiVA.

Hardware Components	Area		Power		Freq.
	[mm^2]	[%]	[mW]	[%]	
MAC Array	72.49	97.24	49831.43	98.37	1.8 GHz
Submap Compressor	1.84	2.47	768.77	1.52	
BNU	0.11	0.15	2.62	0.01	
Row Processing Unit	0.11	0.14	55.72	0.10	
Total	74.55	100	50658.54	100	

6 Evaluations

6.1 Experimental Setup

Evaluation Models and Datasets. We utilize 5 next-resolution prediction models: VAR-d36 [56], Infinity-2B/-8B [15], Switti-AR [59] and HART [53]. For VAR-d36, we use ImageNet [9], for class-conditioned image generation. For Infinity-2B/-8B, HART, and Switti-AR, we use GenEval [13] prompt dataset for text-conditioned image generation. For the text-conditioned models, we report FID [17] against images generated by each original model (with full KV cache) to measure how well each method preserves the original model's output. We also evaluate GenEval (0–100 scale) and ImageReward scores [64], both on 553 GenEval prompts with 4 images each (2,212 samples per method). For class-conditioned VAR-d36, which takes an ImageNet class label and thus cannot be evaluated with GenEval, we compute FID against 50K real ImageNet images.

Baseline Implementations. We compare DiVA against seven baselines: Full KV Fetch, H2O [70], InfiniGen [30], MInference [20], QUEST [54], HACK [43], and ScaleKV [32]. Full KV Fetch fetches each layer's full KV entries without criticality estimation, while H2O performs exact $QK^\top$ scoring for token-wise top-k retrieval. InfiniGen retains the top-30% largest-magnitude channels. MInference applies 32-token average pooling to queries and keys, and QUEST uses 16-token pages [28], reduces the query matrix to a single vector by summing across the token length dimension, and performs $Q{\cdot}K$-based scoring and fetching at the page level.

HACK uniformly samples 32 queries, while ScaleKV selects 16 queries using 2D-spatial patch centroids. We replace their budget allocation policies with per-head retrieval budgets to match DiVA's fetched densities. We denote these retrieval-adapted variants as HACK+R and ScaleKV+R, in Fig. 10.

We adapt H2O, MInference, QUEST, HACK, and ScaleKV to the KV retrieval setting by using their criticality estimation mechanisms to selectively fetch KV entries from CPU memory.

We estimate the next layer's critical submaps using the current layer's query, which is scored against the current layer's key submap representatives. The selected submap indices are then used to prefetch the next layer's KV cache, similar to prior prefetching-based retrievals [22].

We enable KV offloading from stage 6 for VAR-d36 and stage 8 for Infinity, HART, and Switti-AR, where KV cache growth begins to pressure GPU DRAM capacity.

DiVA Implementation. We developed an in-house cycle-level simulator and integrated DRAMSim3 [34]. We model an NVIDIA RTX A6000 GPU with 48 GB of device memory and a DDR4-based host memory of the AMD EPYC™ 7543 processor, and integrate them into our simulator. For PCIe data transfer, we measure the effective bandwidth, latency, and utilization in a real server environment where the processors above are connected by PCIe 4.0 with 32 GB/s bandwidth, and feed these measured values into our simulator. For a fair comparison against baselines, we configure the MAC array of the DiVA system to match the throughput of the Tensor Cores in the NVIDIA RTX A6000 GPU (154.8 TFLOPS). All components of DiVA are designed to operate at the same clock frequency as the A6000 GPU (1.8 GHz). Detailed configurations of DiVA are shown in Table 1. We implement the DiVA core units using Synopsys Design Compiler and synthesize them with the FreePDK 45 nm library [49], while the memory modules are modeled with CACTI [3]. GPU power is measured by NVIDIA-SMI, and PCIe utilization is measured by NVIDIA Nsight Systems [41].

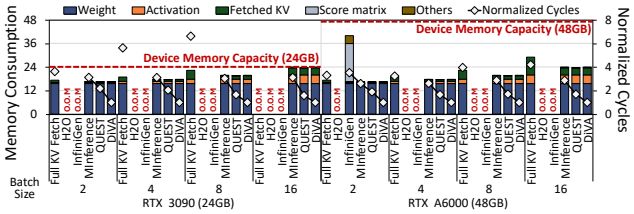


Figure 13: Memory consumption breakdown and execution cycle comparison across batch sizes for Infinity-8B at a retrieval density of 35.75%. Execution cycles are normalized to DiVA in each batch size.

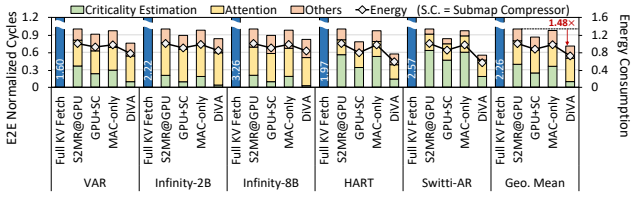
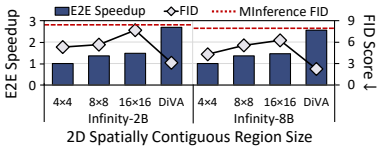


Figure 14: End-to-end cycles breakdown and energy consumption comparison across hardware configurations (normalized to S2MR@GPU).



(a)

Unit	Infinity-2B Pos↑	Infinity-2B All↑	Infinity-8B Pos↑	Infinity-8B All↑
4×4	25.0	67.9	51.0	81.1
8×8	26.0	67.8	50.0	80.9
16×16	26.0	67.1	48.0	80.1
Submap	35.8	68.9	57.0	82.2

(b)

Figure 15: (a) Comparison of speedup and FID score across different spatial contiguous retrieval methods. Speedups are normalized to 4×4 spatial contiguous retrieval of each model. (b) GenEval scores across retrieval granularities, using full-attention probabilities for scoring and varying only the selection granularity.

S2MR is implemented using optimized Triton kernels [57]. Submap reduction is fused into the key-cache write-back path. For each head, the representative vector of each newly generated key submap is computed only once and reused across subsequent decoding stages.

6.2 Performance and Efficiency Analysis

6.2.1 Speedup and Accuracy. We compare the end-to-end speedup and FID score under various retrieval density settings, as shown in Fig. 10. For a fair comparison, we first measure the retrieval density of DiVA for each head and then apply the same density to all baselines. DiVA achieves a 3.26× speedup over Full KV Fetch, and provides speedups of 4.23×, 3.22×, 2.83×, 2.60×, 2.32×, and 1.71× over H2O, InfiniGen, HACK+R, ScaleKV+R, MInference, and QUEST, respectively. H2O and InfiniGen are slower than Full KV Fetch as their token-level criticality estimation incurs great overhead, which exceeds the performance benefits achieved by selective retrieval.

As shown in Fig. 10, DiVA matches or even surpasses H2O at the same retrieval density, despite H2O's exact token-level criticality estimation. DiVA also often achieves comparable accuracy to H2O at a lower retrieval density, enabling higher speedup at a given quality target. DiVA also outperforms all other baselines, and even achieves accuracy comparable to H2O (w/o prefetch), which performs exact token-level estimation without any approximation or next-layer prediction, indicating that preserving the complete spatial structure of each submap is more effective for KV retrieval in VAR than fine-grained token-level importance. As shown in Table 2, GenEval and ImageReward scores also consistently support that DiVA preserves image quality close to Full KV Fetch while outperforming other retrieval baselines.

6.2.2 Energy Efficiency. As shown in Fig. 11, we also compare the energy consumption breakdown of the DiVA system against baselines. DiVA improves energy efficiency by 3.02×, 2.35×, 2.03×, and 1.66× over H2O, InfiniGen, MInference, and QUEST. This improvement comes from reductions in both computation and memory access energy. DiVA reduces core unit energy by reducing the execution cycles of the criticality estimation stage. Compared with H2O, this reduces core energy by 81.24%. Second, submap-granularity retrieval allows DiVA to access contiguous KV cache chunks (i.e., submaps) instead of irregularly scattered token-level entries. This improves DRAM access efficiency and reduces DRAM energy by 71.87%. Finally, the submap-wise access pattern also improves SRAM locality, reducing SRAM energy by 66.24% compared with H2O.

6.2.3 PCIe Utilization. We compare the effective data transfer rate and PCIe utilization ratio. Fig. 12a presents the effective PCIe data transfer rate, computed as transferred data size divided by transfer latency. DiVA reduces PCIe idle time by lowering criticality estimation latency. Submap-level fetching enables larger contiguous fetches, improving the effective transfer rate, resulting in a denser transfer pattern over cycles than InfiniGen and QUEST. Fig. 12b shows the PCIe utilization ratio, computed as the fraction of PCIe-utilized cycles over total cycles. DiVA achieves 61.31%, higher than the 13.47% of InfiniGen and the 32.99% of QUEST. Results imply that DiVA uses PCIe bandwidth more efficiently than baselines, minimizing PCIe idle time and allowing PCIe transfers to overlap more effectively with other operations.

6.3 Area and Power Analysis

Table 3 reports the area and power breakdown of DiVA. S2MR engine, which consists of Submap Compressor, BNU, and RPU, occupies 2.06 mm^2, accounting for 2.76% of the total area. Its power consumption is 827.11 mW, corresponding to 1.63% of the total system power. Score Buffer stores submap scores using 1 byte per submap, and requires 3 KB of SRAM, which is negligible compared with the SRAM used by the MAC array matched to the throughput of an A6000 GPU (over 10 MB). Results show that integrating the S2MR engine incurs reasonable area, power, and memory overhead.

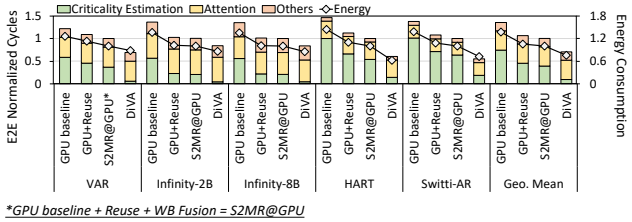


Figure 16: Impact of GPU optimizations on end-to-end latency and energy consumption. GPU baseline indicates the PyTorch implementation.

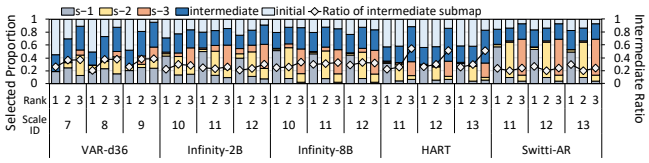


Figure 17: Distribution of historical submaps selected by DiVA across decoding stages. Initial refers to submaps from the first two scales. s-1, s-2, and s-3 refer to the first (i.e., most), second, and third recent scales, while intermediate refers to the remaining scales between initial and s-3.

6.4 Design Space Exploration

6.4.1 Batch Variation. We evaluate the memory consumption breakdown and latency under 24 and 48 GB (matching the memory capacity of RTX 3090 and A6000), as shown in Fig. 13. H2O and InfiniGen require materializing a large intermediate score matrix during criticality estimation, causing OOM errors under a 24 GB configuration even at batch size 2. At batch size 2 under 48 GB, both materialize the same score matrix, but InfiniGen's channel reduction keeps its peak memory consumption below the limit by reducing intermediate memory, while H2O does not. Among methods within given memory budgets, DiVA achieves the lowest latency while preserving the best accuracy, as discussed in Fig. 10.

6.4.2 Impact of Each Design Component. To evaluate each component's contribution, we compare Full KV Fetch, S2MR@GPU, GPU+SC, MAC-only, and DiVA (Fig. 14). GPU+SC offloads only submap compression to the Submap Compressor, leaving other S2MR operations on the GPU. MAC-only runs S2MR on DiVA's MAC array without the S2MR Engine. Although S2MR@GPU achieves a 2.26× E2E speedup over Full KV Fetch, criticality estimation still accounts for 39% of its E2E runtime on average. GPU+SC achieves a 1.17× E2E speedup over S2MR@GPU by offloading submap compression, but representative scoring (on GPU) leaves criticality estimation accounting for 28.5% of its E2E runtime. DiVA accelerates representative scoring using the RPU and BNU with submap-stationary dataflow, reducing its share to 11.3%. DiVA achieves a 1.45× E2E speedup and a 1.41× energy-efficiency improvement over MAC-only (indicating a gain from the S2MR Engine). Compared with S2MR@GPU, DiVA's criticality estimation alone achieved 5.06× speedup, with 1.48× end-to-end speedup and 1.43× energy efficiency, showing combined benefit of each component.

6.4.3 GPU baseline optimization. To show the impact of GPU kernel optimizations and DiVA's performance, we progressively optimize a PyTorch S2MR implementation (GPU baseline) and compare its E2E latency and energy consumption with those of DiVA (Fig. 16). GPU+Reuse computes each submap representative once when the submap is created and reuses it across subsequent stages, eliminating repeated compression. S2MR@GPU additionally fuses submap compression within the KV cache write-back path of newly generated keys, reducing redundant memory accesses. S2MR@GPU achieves 1.39× and 1.11× E2E speedups over GPU baseline and GPU+Reuse, respectively. DiVA achieves an extra 1.48× E2E speedup and a 1.43× energy-efficiency improvement over optimized S2MR@GPU. These suggest that DiVA's specialized engine and dataflow, tailored to S2MR's asymmetric operand reuse, can reduce the overhead that remains after GPU optimizations.

6.4.4 Intra-submap Granularity Sweep. To quantify the benefits of spatial locality, we compare submap-level retrieval with smaller 2D spatially contiguous retrieval units (4×4, 8×8, and 16×16) at matched per-head retrieval densities (Fig. 15). All spatially contiguous methods outperform MInference's 1D pooling, suggesting benefits from preserving 2D spatial structure. However, finer regions yield worse FID than submap-level retrieval despite their greater selection flexibility, potentially due to fragmentation of complete submaps. Moreover, their smaller pooling units retain 256×, 64×, and 16× higher scoring complexity, limiting speedup to 1.00/1.35/1.49×, compared with DiVA's 2.68× on Infinity-2B.

To evaluate the effect of selection granularity itself, we compute each retrieval unit's score with full-attention probabilities and vary only the retrieval granularity under identical KV-byte budgets (Fig. 15b). Submap-level retrieval achieves the best generation quality, indicating that preserving the spatial completeness of each submap is crucial for the final generation quality.

6.5 Submap Selection Analysis

6.5.1 Dynamic Submap Selection. Figure 17 shows the distribution of selected historical submaps. Initial- and recent-scale submaps are frequently selected, but intermediate-scale submaps still account for at least 20% of selections in every evaluated stage, indicating that critical submaps are spread across scales and that retaining only fixed scales can cause significant accuracy loss.

7 Conclusion

DiVA is the first co-designed architecture for KV cache retrieval in VAR models, to our knowledge. S2MR shifts the retrieval granularity from tokens to submaps and effectively reduces the criticality estimation overhead. To efficiently support S2MR, DiVA incorporates the S2MR Engine for submap compression, row-wise processing, and lightweight normalization. DiVA achieves up to 4.23× end-to-end speedup and 3.02× energy-efficiency improvement while maintaining generation quality close to ideal token selection.

References

[1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems* 6 (2024), 114–127.

[2] Hyesung Ahn, Ranggi Hwang, and Minsoo Rhu. 2025. Exploring KV Cache Quantization in Multimodal Large Language Model Inference. *IEEE Computer Architecture Letters* 25, 1 (2025), 13–16.

[3] Rajeev Balasubramonian, Andrew B Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New tools for interconnect exploration in innovative off-chip memories. *ACM Transactions on Architecture and Code Optimization (TACO)* 14, 2 (2017), 1–25.

[4] Abhimanyu R Bambhaniya, Sheng-Chun Kao, Geonhwa Jeong, Suvinay Subramanian, Amir Yazdanbaksh, and Tushar Krishna. 2026. Demystifying the cost versus benefits of sparse LLM acceleration. *IEEE Micro* (2026).

[5] Jonathan Cederlund, Axel Berg, William Isaksson, Durmus Alp Emre Acar, Chuteng Zhou, and Pontus Giselsson. 2026. HeatKV: Head-Tuned KV-Cache Compression for Visual Autoregressive Modeling. arXiv:2605.14877 [cs.CV]

[6] Stefano Ceri, Alessandro Bozzon, Marco Brambilla, Emanuele Della Valle, Piero Fraternali, and Silvia Quarteroni. 2013. An Introduction to Information Retrieval. In *Web Information Retrieval*. Springer, Berlin, Heidelberg, 3–11. doi:10.1007/978-3-642-39314-3_1

[7] Deming Chen, Jason Cong, Azalia Mirhoseini, Christos Kozyrakis, Subhasish Mitra, Jinjun Xiong, Cliff Young, Anima Anandkumar, Michael Littman, Aron Kirschen, Sophia Shao, Serge Leef, Naresh Shanbhag, Dejan Milojicic, Michael Schulte, Gert Cauwenberghs, Jerry M. Chow, Tri Dao, Kailash Gopalakrishnan, Richard Ho, Hoshik Kim, Kunle Olukotun, David Z. Pan, Mark Ren, Dan Roth, Aarti Singh, Yizhou Sun, Yusu Wang, Yann LeCun, and Ruchir Puri. 2026. AI+ HW 2035: Shaping the Next Decade. *arXiv preprint arXiv:2603.05225* (2026).

[8] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. 2025. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 187–201.

[9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. *2009 IEEE conference on computer vision and pattern recognition* (2009), 248–255.

[10] Peiyan Dong, Jinming Zhuang, Zhuoping Yang, Shixin Ji, Yanyu Li, Dongkuan Xu, Heng Huang, Jingtong Hu, Alex K. Jones, Yiyu Shi, Yanzhi Wang, and Peipei Zhou. 2024. EQ-ViT: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on Versal ACAP architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 11 (2024), 3949–3960.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations* (2021).

[12] Ravi Ghadia, Avinash Kumar, Gaurav Jain, Prashant Nair, and Poulami Das. 2025. Dialogue without limits: Constant-sized KV caches for extended responses in LLMs. *arXiv preprint arXiv:2503.00979* (2025).

[13] Dhruba Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. 2023. Geneval: An object-focused framework for evaluating text-to-image alignment. *Advances in Neural Information Processing Systems* 36 (2023), 52132–52152.

[14] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. 2025. Pim is all you need: A cxl-enabled gpu-free system for large language model inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 862–881.

[15] Jian Han, Jinlai Liu, Yi Jiang, Bin Yan, Yuqi Zhang, Zehuan Yuan, Bingyue Peng, and Xiaobing Liu. 2025. Infinity: Scaling bitwise autoregressive modeling for high-resolution image synthesis. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 15733–15744.

[16] Jitai Hao, Yuke Zhu, Tian Wang, Jun Yu, Xin Xin, Bo Zheng, Zhaochun Ren, and Sheng Guo. 2025. OmniKV: Dynamic Context Selection for Efficient Long-Context LLMs. In *The Thirteenth International Conference on Learning Representations*.

[17] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems* 30 (2017).

[18] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Yakun Sophia Shao. 2025. Speed: Speculative pipelined execution for efficient decoding. In *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*. Springer, 19–32.

[19] Nikoleta Iliakopoulou, Jovan Stojkovic, Chloe Alverti, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2025. Chameleon: Adaptive caching and scheduling for many-adapter llm inference environments. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 217–231.

[20] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Advances in Neural Information Processing Systems* 37 (2024), 52481–52515.

[21] Donghyeon Joo, Helya Hosseini, Ramyad Hadidi, and Bahar Asgari. 2025. Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. *arXiv preprint arXiv:2505.22913* (2025).

[22] Donghyuk Kim, Sejeong Yang, Wonjin Shin, and Joo-Young Kim. 2025. V-rex: Real-time streaming video llm acceleration via dynamic kv cache retrieval. *arXiv preprint arXiv:2512.12284* (2025).

[23] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W Lee, Sangdoo Yun, and Hyun Oh Song. 2025. Kvzip: Query-agnostic kv cache compression with context reconstruction. *arXiv preprint arXiv:2505.23416* (2025).

[24] Yujin Kim, Faraz Tahmasebi, Gunjae Koo, and Hyoukjun Kwon. 2026. LOGFLEX: Flexible-bit Log Arithmetic Accelerator for Language Models on Edge. *IEEE Micro* (2026).

[25] Anna Kuzina, Maciej Pióro, and Babak Ehteshami Bejnordi. 2026. KaVa: Latent Reasoning via Compressed KV-Cache Distillation. In *The Fourteenth International Conference on Learning Representations*.

[26] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Gyeonggeun Jung, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yosub Song, Byeongsu Yang, Haerang Choi, Guhyun Kim, Jongsoon Won, Woojae Shin, Changhyun Kim, Gyeongcheol Shin, Yongkee Kwon, Ilkon Kim, Euicheol Lim, John Kim, and Jungwook Choi. 2026. PIMphony: Overcoming Bandwidth and Capacity Inefficiency in PIM-Based Long-Context LLM Inference System. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–21.

[27] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yosub Song, Byeongsu Yang, Haerang Choi, Guhyun Kim, Jongsoon Won, Woojae Shin, Changhyun Kim, Gyeongcheol Shin, Yongkee Kwon, Ilkon Kim, Euicheol Lim, John Kim, and Jungwook Choi. 2024. LoL-PIM: Long-context LLM decoding with scalable DRAM-PIM system. *arXiv preprint arXiv:2412.20166* (2024).

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP '23: Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626.

[29] Sanghyeon Lee, Hongbeen Kim, Soojin Hwang, Guseul Heo, Minwoo Noh, and Jaehyuk Huh. 2025. Throughput-Oriented LLM Inference via KV-Activation Hybrid Caching with a Single GPU. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 269–276.

[30] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.

[31] Cong Li, Yihan Yin, Chenhao Xue, Zhao Wang, Fujun Bai, Yixin Guo, Xiping Jiang, Qiang Wu, Yuan Xie, and Guangyu Sun. 2026. Hardware-Software Co-design for 3D-DRAM-based LLM Serving Accelerator. *arXiv preprint arXiv:2603.04797* (2026).

[32] Kunjun Li, Zigeng Chen, Cheng-Yen Yang, and Jenq-Neng Hwang. 2025. Memory-Efficient Visual Autoregressive Modeling with Scale-Aware KV Cache Compression. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

[33] Ruihao Li, Shagnik Pal, Vineeth Narayan Pullu, Prasoon Sinha, Jeeho Ryoo, Lizy K John, and Neeraja J Yadwadkar. 2025. Oneiros: KV Cache Optimization through Parameter Remapping for Multi-tenant LLM Serving. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 88–101.

[34] Shougang Li, Zhiying Yang, Dinesh Reddy, Anirudh Srivastava, and Bruce Jacob. 2020. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.

[35] Tianhong Li, Dina Katabi, and Kaiming He. 2023. Self-conditioned image generation via generating representations. *arXiv preprint arXiv:2312.03701* (2023).

[36] Chaofan Lin, Jiaming Tang, Shuo Yang, Hanshuo Wang, Tian Tang, Boyu Tian, Ion Stoica, Song Han, and Mingyu Gao. 2025. Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning. In *Advances in Neural Information Processing Systems*, Vol. 38.

[37] Guangda Liu, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Danning Ke, Minyi Guo, and Jieru Zhao. 2025. FreeKV: Boosting KV Cache Retrieval for Efficient LLM Inference. *arXiv preprint arXiv:2505.13109* (2025). arXiv:2505.13109 [cs.LG] https://arxiv.org/abs/2505.13109

[38] Seungjae Moon, Junseo Cha, Hyunjun Park, and Joo-Young Kim. 2025. Hybe: Gpu-npu hybrid system for efficient llm inference with million-token context window. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 808–820.

[39] Seonjin Na, Geonhwa Jeong, Byung H Ahn, Aaron Jezghani, Jeffrey Young, Christopher J Hughes, Tushar Krishna, and Hyesoon Kim. 2025. Flexinfer: Flexible llm inference with cpu computations. *Proceedings of Machine Learning and Systems* 7 (2025).

[40] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. 2024. Understanding performance implications of LLM inference on CPUs. In *2024 IEEE international symposium on workload characterization*

(IISWC). IEEE, 169–180.

[41] NVIDIA. 2026. NVIDIA Nsight Systems. https://developer.nvidia.com/nsight-systems Accessed: 2026-03-27.

[42] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 4195–4205.

[43] Ziran Qin, Youru Lv, Mingbao Lin, Hang Guo, Zeren Zhang, Danping Zou, and Weiyao Lin. 2026. Head-Aware KV Cache Compression for Efficient Visual Autoregressive Modeling. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 30 (2026), 24982–24990. doi:10.1609/aaai.v40i30.39686

[44] Derrick Quinn, E Ezgi Yücel, Jinkwon Kim, José F Martínez, and Mohammad Alian. 2025. LongSight: Compute-Enabled Memory to Accelerate Large-Context LLMs via Sparse Attention. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 34–48.

[45] Akshat Ramachandran, Souvik Kundu, Arnab Raha, Shamik Kundu, Deepak K Mathaikutty, and Tushar Krishna. 2025. Accelerating LLM inference with flexible N: M sparsity via a fully digital compute-in-memory accelerator. In *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–7.

[46] Michael Shen, Muhammad Umar, Kiwan Maeng, G Edward Suh, and Udit Gupta. 2025. Hermes: Algorithm-system co-design for efficient retrieval-augmented generation at-scale. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 958–973.

[47] Jiuchen Shi, Hang Zhang, Yixiao Wang, Quan Chen, Yizhou Shan, Kaihua Fu, Wei Wang, and Minyi Guo. 2026. ELORA: Efficient LoRA and KV Cache Management for Multi-LoRA LLM Serving. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–14.

[48] Jacob R Stevens, Rangharajan Venkatesan, Steve Dai, Brucek Khailany, and Anand Raghunathan. 2021. Softermax: Hardware/software co-design of an efficient softmax for transformers. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 469–474.

[49] James E. Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajaiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An open-source variation-aware design kit. In *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*. IEEE, 173–174.

[50] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-throughput llm inference via model re-sharding. *Proceedings of Machine Learning and Systems* 7 (2025).

[51] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. 2025. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 57355–57373.

[52] Peize Sun, Yifan Jiang, Shoufa Chen, Shengyu Zhang, Bo Peng, Ping Luo, and Zehuan Yuan. 2024. Autoregressive model beats diffusion: LLaMA for scalable image generation. *arXiv preprint arXiv:2406.06525* (2024).

[53] Haotian Tang, Yuxin Wu, Shuai Yang, Enze Xie, Jiahao Chen, Jingwen Chen, and Song Han. 2024. Hart: Efficient visual generation with hybrid autoregressive transformer. *arXiv preprint arXiv:2410.10812* (2024).

[54] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*. 47901–47911.

[55] Tian Tang, Kan Zhu, Qinyu Xu, Zhan Jin, Yile Gu, Zhichen Zeng, Rohan Kadekodi, Liangyu Zhao, Ang Li, Arvind Krishnamurthy, and Baris Kasikci. 2026. Tactic: Adaptive Sparse Attention with Clustering and Distribution Fitting for Long-Context LLMs. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=tJod11fK1A

[56] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. 2024. Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction. *Advances in neural information processing systems* 37 (2024), 84839–84865.

[57] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[58] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[59] Anton Voronov, Denis Kuznedelev, Mikhail Khoroshikh, Valentin Khrulkov, and Dmitry Baranchuk. 2024. Switti: Designing Scale-Wise Transformers for Text-to-Image Synthesis. *arXiv preprint arXiv:2412.01819*.

[60] Huizheng Wang, Zichuan Wang, Zhiheng Yue, Yousheng Long, Taiquan Wei, Jianxun Yang, Yang Wang, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. MCBP: A memory-compute efficient LLM inference accelerator leveraging bit-slice-enabled sparsity and repetitiveness. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1592–1608.

[61] Xinkai Wang, Yiming Zhuansun, Chao Li, Jing Wang, Xiaofeng Hou, Lingyu Sun, Luping Wang, and Minyi Guo. 2025. Asymserve: Demystifying and optimizing llm serving efficiency on cpu acceleration units. In *International Symposium on Advanced Parallel Processing Technologies*. Springer, 231–245.

[62] Yutong Wang, Haiyu Wang, and Sai Qian Zhang. 2025. QSVD: Efficient Low-rank Approximation for Unified Query-Key-Value Weight Compression in Low-Precision Vision-Language Models. *arXiv preprint arXiv:2510.16292* (2025).

[63] Boxun Xu, Yu Wang, Zihu Wang, and Peng Li. 2026. AMS-KV: Adaptive KV Caching in Multi-Scale Visual Autoregressive Transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 27206–27214. doi:10.1609/aaai.v40i32.39936

[64] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. 2023. ImageReward: learning and evaluating human preferences for text-to-image generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 15903–15935.

[65] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1–17.

[66] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Chendi Li, Jinghua Yan, Yu Bai, Ponnuswamy Sadayappan, Xia Hu, and Bo Yuan. 2025. TopV: Compatible token pruning with inference time optimization for fast and low-memory multimodal vision language model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19803–19813.

[67] Haoyu Yang, Qijing Huang, Nathaniel Pinckney, Walker Turner, Wenfei Zhou, Yanqing Zhang, Chia-Tung Ho, Chen-Chia Chang, and Haoxing Ren. 2025. ChipVQA: Benchmarking Visual Language Models for Chip Design. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–6.

[68] Zebin Yang, Tong Xie, Baotong Lu, Shaoshan Liu, Bo Yu, and Meng Li. 2026. KEEP: A KV-Cache-Centric Memory Management System for Efficient Embodied Planning. *arXiv preprint arXiv:2602.23592* (2026).

[69] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).

[70] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 34661–34710.

[71] Jiaqi Zhou, Hongou Li, Keyao Jiang, Yiyang Sun, and Tianyu Jia. 2026. 31.4 VARSA: A Visual Autoregressive Generation Accelerator Using Performance-Scalable Multi-Precision PE-LUT and Grid-Similarity Attention Compression. In *2026 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 69. IEEE, 538–540.

[72] Zeyu Zhu, Gang Li, Tielong Liu, Zitao Mo, Xiaoyao Liang, and Jian Cheng. 2025. MATA: A Memory-Efficient Attention Accelerator for LLMs Exploiting Look-Back KV Cache Pruning. *IEEE Trans. Comput.* (2025).

[73] Jing Zou, Shangyu Wu, Hancong Duan, Qiao Li, and Chun Jason Xue. 2026. ContiguousKV: Accelerating LLM Prefill with Granularity-Aligned KV Cache Management. *arXiv preprint arXiv:2601.13631* (2026). arXiv:2601.13631 [cs.OS]